

面向中国象棋的人机对弈机器人系统设计与实现

——基于视觉识别、Pikafish 引擎与 STM32 运动控制

Maicarons

2026 年 8 月

摘 要

中国象棋 (Xiangqi) 是世界上参与人数最多的棋类运动之一, 其人机对弈长期以来依赖纯软件引擎: 引擎在屏幕上输出着法, 由人类完成物理落子。将“感知—决策—执行”三者闭环到一台物理机器人上, 需要同时解决视觉棋盘识别、棋局智能决策与机械臂精确移子等跨学科难题, 是具身智能 (embodied AI) 在棋类场景中的典型体现。

本文系统介绍 CH-RO (Chinese Chess Robot) 对弈机器人系统: 一套以 Python 服务、Web 操作界面、UCI 中国象棋引擎与 STM32 下位机固件构成的完整闭环。在感知层, 系统建立摄像机针孔成像模型, 采用 RTMPose 四角关键点检测 (底层为 SimCC 坐标分类范式) 与透视变换完成棋盘定位, 并以单一全局分类器 ONNX 模型一次性输出整盘 90 个交叉点的类别 (14 种棋子 + 空 + 未识别), 避免了逐格检测与目标检测器的调度与后处理开销; 多帧稳定投票与基于象棋规则的动态走子推断进一步提升了鲁棒性。在认知层, 系统通过 UCI 协议驱动 Pikafish 引擎 (UCI_Variant 设为 xiangqi), 以扩展 FEN 表示棋局状态; 本文对 alpha-beta 剪枝、迭代加深、静态搜索与 NNUE 神经评估等由引擎提供的搜索原理作了说明, 并明确这些算法位于所调用的引擎内部, 本系统承担的是协议驱动与结果解析职责。在执行层, 系统定义了一种仅含五个整数的 ASCII 指令协议 (`startX,startY,endX,endY,signal`), 经 TCP 端口 8086 下发至 STM32 (ESP8266 透传) 下位机, 由下位机完成 SCARA 逆运动学解算与气泵抓取; 完成确认以 “STATE:5,RESULT:1” 握手实现, 且运动指令非幂等、缺失确认即上报而不会自动重发。接口层基于 Flask 提供实时视频、单步/连续识别、硬件与仿真双模式对弈的人机交互。论文逐层给出模块划分、识别流水线、协议字段、坐标映射公式与下位机运动学, 并以真实代码规模、模型规格与测试覆盖数据说明工程实现与质量保障。整套系统在解耦、可测试的纯函数工具包与组合根架构下运行, 可为象棋机器人及类似的“视觉—决策—执行”具身智能系统提供参考。

关键词 中国象棋; 人机对弈; 机器人; 视觉识别; RTMPose; SimCC; UCI 协议; Pikafish; NNUE; STM32; SCARA; ONNX

目录

第一章 绪 论	7
1.1 研究背景与意义	7
1.2 国内外研究现状	7
1.3 问题与挑战	8
1.4 研究目标与主要贡献	8
1.5 论文结构	9
第二章 系统总体架构与数据契约	10
2.1 四层架构与数据契约	10
2.2 模块职责与解耦策略	10
2.3 运行时组合与依赖关系	11
第三章 视觉感知与棋盘识别	13
3.1 图像采集与摄像机模型	13
3.2 棋盘定位：四角关键点与透视变换	13
3.2.1 单应矩阵与透视校正原理	14
3.3 整盘棋子分类	14
3.4 多帧稳定与动态走子推断	14
3.5 真实棋盘图像示例	15
第四章 棋局决策引擎与棋局表示	17
4.1 PikaFish 与 UCI 协议	17
4.2 搜索引擎原理（引擎侧）	17
4.3 中国象棋 FEN 与坐标表示	18
4.4 着法记谱转换	18
第五章 机械臂控制与通信协议	19
5.1 五值 ASCII 指令协议	19
5.2 棋盘坐标到机械臂坐标的映射	19
5.3 持久化 TCP 客户端与确认握手	20
5.4 STM32 下位机实现与运动学	21

第六章 软件编排与 Web 仿真	23
6.1 Flask 组合根与蓝图架构	23
6.2 领域纯函数与状态表示	23
6.3 对局状态机与双模式	23
6.4 HTTP API 与前端交互	24
第七章 工程实现与质量保障	25
7.1 代码规模与解耦	25
7.2 测试体系	25
7.3 ONNX 推理与执行提供者	26
第八章 实验与系统指标	27
第九章 结论与展望	28

插图

2.1	CH-RO 系统四层架构。感知层将物理棋盘转为 FEN；认知层生成走法；执行层驱动机械臂；接口层以 Web 方式贯穿各层并提供人机交互。	10
2.2	组合根（app.py）通过 services.py 的懒加载工厂解析各层依赖；domain.py 为不依赖 I/O 的纯函数，便于独立测试。	12
3.1	视觉识别流水线：从原始图像经四角关键点定位、透视校正，到全局分类与 FEN 生成，再由多帧稳定模块对连续帧去噪并推断人类走子。	13
3.2	DynamicBoardTracker 的状态机：以首张稳定棋盘为基线，依据占用变化推断走子或判为噪声。	15
3.3	系统采集的真实棋盘图像示例（调试用）。经 RTMPose 四角定位与透视校正后送入全局分类器。	15
5.1	棋盘几何与坐标映射示意。原点位于右上角（黑方右车），列间距 34mm、普通行间距 30mm、河界行间距 32mm。	20
5.2	上位机与 STM32 的通信时序：连接后先归位握手，再逐条下发五值移子指令并等待完成确认。	21

表格

2.1	系统主要模块及其职责	11
3.1	全局分类器输出的 16 个类别（红方大写、黑方小写）	14
4.1	中国象棋 FEN 棋子字符与基础属性	18
5.1	五值指令协议字段语义	19
7.1	主要模块代码规模（实测）	25
7.2	测试固化的关键不变量（节选）	26
8.1	模型与权重规格（仓库真实文件）	27
8.2	关键配置常量（取自 <code>config.py</code> ）	27

第一章 绪 论

1.1 研究背景与意义

中国象棋是一种在 9 列 $\times$ 10 行交叉点棋盘上进行的两人对弈棋类游戏，棋盘中央以“河界”分隔双方领地，并在两端设有“九宫”等规则约束^[1]。作为与西洋棋并列的最具影响力的棋类之一，计算机中国象棋历经数十年发展，已出现以象眼（ElephantEye）、Pikafish 等为代表的强引擎；其中 Pikafish 是从 Stockfish 派生、面向中国象棋的免费开源 UCI 引擎，采用 NNUE 神经评估函数，是当前最强的开源中国象棋引擎之一^[2-3]。然而，绝大多数相关研究停留在纯软件层面：引擎在屏幕上输出着法，由人类落子。将决策结果转化为物理世界中的真实移子动作，需要构建“摄像头识别棋盘—引擎决策—机械臂执行”的闭环系统，这正是具身智能在棋类场景中的典型体现，对服务机器人、教育机器人与人机协作均有借鉴意义。

1.2 国内外研究现状

计算机棋类博弈是人工智能的经典试验场。在西洋棋方向，深蓝（Deep Blue）于 1997 年首次在标准时限下战胜国际象棋世界冠军，其基于大规模并行硬件与手工评估函数的搜索体系奠定了后续工程范式^[4]；而强化学习方法的引入则进一步改变了范式——TD-Gammon 用时序差分学习直接从自我对弈中习得西洋棋后端博弈策略^[5]，KnightCap 将 TD(λ) 与博弈树搜索结合提升棋力^[6]，直至 AlphaZero 以通用强化学习算法在自我对弈中同时掌握国际象棋、将棋与围棋，展示了“搜索 + 深度表征”的通用性^[7]。在算法层面，alpha-beta 剪枝、迭代加深与静态搜索（quiescence search）仍是现代引擎的基石，并在《人工智能：一种现代方法》等教材中有系统论述^[8]。

在引擎接口层面，通用国际象棋接口 UCI（Universal Chess Interface）由 Meyer-Kahlen 等人规范，以标准输入/输出文本指令通信^[9]；中国象棋则有借鉴 UCI 的 UCCI（Universal Chinese Chess Interface）协议^[10]与扩展棋局表示 XQFEN^[11]，使不同引擎可在统一协议下被调用。Pikafish 在实现上复用 UCI 框架，并通过 UCI_Variant 选项支持中国象棋变体^[2]。自 2018 年 Nasu 提出 NNUE（Efficiently Updatable Neural-Network-based Evaluation）以来，神经评估逐渐取代手工评估，成为开源引擎的主流^[12]。

在视觉感知层面，棋类棋盘识别经历了从传统方法到深度学习的演进。早期工作依赖棋盘纹理的规则性，使用 OpenCV 的 findChessboardCorners 检测内角点完成定位^[13]；但中国象棋棋盘存在文字棋子遮挡、纹理不完整等问题，纯角点法鲁棒性不足。近年来，基于关键点的方案展现出优势：RTMPose 是 OpenMMLab MMPose 提出的实时多人姿态估计框架，在 COCO 上以 90+ FPS 取得 75.8% AP，可在移动端部署^[14-15]；其底层采用 SimCC（Simple Coordinate

Classification) 关键点范式, 用“坐标分类”替代高斯热图回归, 实现高精度与低延迟^[16], 非常契合约束场景下的棋盘四角定位。在棋子识别上, 通用目标检测器 (如 YOLO^[17]) 虽通用, 但逐子检测存在调度与后处理开销; 而受 ImageNet 预训练与深度卷积网络 (AlexNet^[18]、ImageNet^[19]) 启发的“整盘一次性分类”思路, 则更适合边缘部署。模型部署侧, ONNX 提供了开放的神经网络交换格式^[20], ONNX Runtime 则提供跨平台、支持 CUDA/DML/CPU 多种执行提供者的高性能推理后端^[21], 便于将训练好的模型部署到异构硬件。

在机械执行层面, 棋类机器人早期以西洋棋机械臂为主, 核心难点在于棋子抓取规划与视觉定位。SCARA 等平面关节机器人的运动学建模与控制已有成熟教材体系^[22-23], 而 STM32 等低成本微控制器凭借丰富的定时器与通信外设, 常作为下位机承担逆解与电机/气泵时序^[24]。本文工作正是在上述“强引擎 + 关键点视觉 + 边缘推理 + 低成本下位机”的技术成熟度基础上, 构建一套完整、可解耦、可测试的中国象棋对弈机器人。

1.3 问题与挑战

将软件博弈闭环到物理机器人, 面临三方面挑战:

- (1) **视觉识别的鲁棒性:** 真实桌面存在光照变化、棋子文字遮挡、视角倾斜与镜头畸变, 需在边缘算力下以低延迟输出准确棋局。
- (2) **决策与物理执行的对齐:** 引擎输出的是逻辑着法 (UCI 坐标), 必须精确映射到机械臂坐标系并安全执行; 任何坐标错配都会导致抓错子或碰子。
- (3) **系统可维护性与可测试性:** 视觉、决策、控制三者耦合度高, 若缺乏清晰的数据契约与解耦架构, 难以独立调试与回归测试。

1.4 研究目标与主要贡献

本文目标是给出一套可解耦、可测试、可联调的中国象棋对弈机器人完整实现, 并就其关键设计给出严谨描述。主要贡献如下:

- (1) 提出“感知—认知—执行—接口”四层系统架构, 并以纯函数工具包、命令行包、Web 组合根等方式实现模块解耦 (模块规模见表 7.1), 使视觉、决策、控制可独立测试。
- (2) 设计轻量棋盘识别方案: 建立摄像机针孔模型, 以 RTMPose 四角关键点完成透视校正, 再以单一全局分类器一次性输出全棋盘 90 个交叉点类别, 避免逐格检测与目标检测器的推理开销。
- (3) 定义并实现一种仅含五个整数的 ASCII 五值指令协议与持久化 TCP 客户端, 配合 STM32 下位机的运动学解算与“STATE:5,RESULT:1”完成确认握手, 保证走子的确定性。
- (4) 提供硬件/仿真双模式与基于 Flask 的 Web 操作界面, 使系统在无实体机械臂时亦可完成端到端联调。

1.5 论文结构

第 二 章给出系统总体架构与层间数据契约；第 三 章详述视觉识别流水线及其数学原理；第 四 章介绍决策引擎、搜索原理与棋局表示；第 五 章阐述机械臂控制协议、坐标映射、TCP 握手与固件实现；第 六 章说明软件编排与 Web 仿真；第 七 章给出工程实现与质量保障数据；第 八 章汇总真实系统指标；第 九 章总结并展望未来工作。

本章小结

本章从棋类博弈与具身智能的背景出发，梳理了引擎、协议、视觉与执行四个方向的研究现状，明确了将软件博弈闭环到物理机器人所面临的三大挑战，并由此引出本文的四层架构设计与四项主要贡献。后续各章将逐层展开：先给出总体架构与数据契约，再依次深入感知、认知、执行与接口四个层次，最后以工程实现与系统指标佐证方案的可行性与质量。

第二章 系统总体架构与数据契约

CH-RO 系统由感知、认知、执行与接口四个层次构成，各层之间仅通过明确的数据契约（图像帧、FEN 字符串、五值指令、ACK 报文）耦合，从而支持独立替换与测试。整体架构如图 2.1 所示。

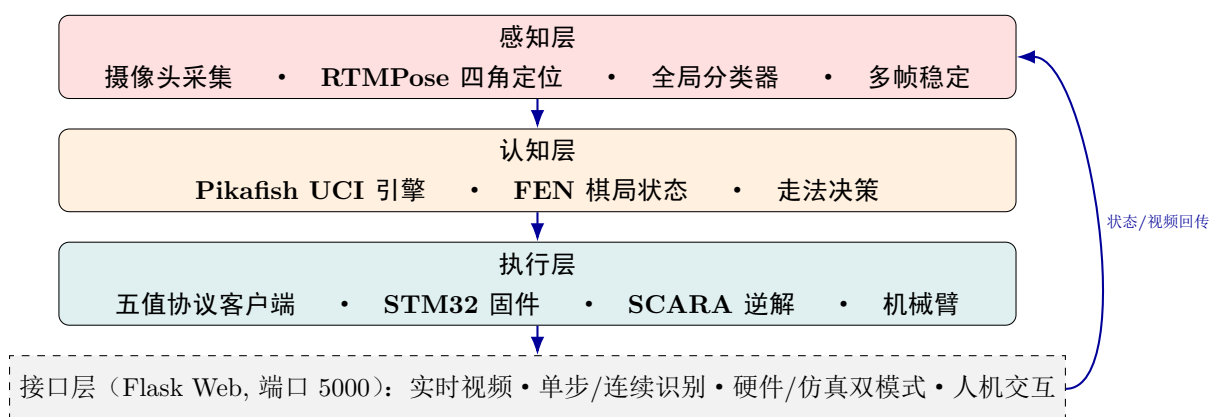


图 2.1: CH-RO 系统四层架构。感知层将物理棋盘转为 FEN；认知层生成走法；执行层驱动机械臂；接口层以 Web 方式贯穿各层并提供人机交互。

2.1 四层架构与数据契约

- **感知层**：输入为图像帧，输出为 FEN 字符串。其职责是“看见棋盘”。
- **认知层**：输入为 FEN，输出为 UCI 着法（如 h2e2）。其职责是“想出下一步”。
- **执行层**：输入为 UCI 着法，输出为五值指令与机械臂动作，并最终 ACK 报文确认完成。
- **接口层**：横向贯穿三层，提供人机交互、状态可视与模式切换。

四层之间不共享可变全局状态（除 Web 组合根持有受控的共享字典外），而是通过纯函数（FEN 解析、坐标映射）与消息（五值指令、ACK）通信，这是系统可测试性的根本来源。

2.2 模块职责与解耦策略

系统的主要 Python 包及其职责如表 2.1 所示。其中 `xq_utils` 是由原根目录 `utils.py` 拆分出的纯函数包，集中实现 FEN、坐标与记谱工具；`web_simulation` 以组合根（composition root）加蓝图（Blueprint）的方式组织 Flask 应用，是当前激活的硬件控制路径。

表 2.1: 系统主要模块及其职责

模块	关键类/函数	职责
xq_utils	FENUtils, CoordinateUtils, MoveNotationUtils, BoardUtils	中国象棋 FEN 解析/生成、棋盘坐标与 UCI 互转、简化 WXF/中文记谱
vision	BoardRecognizer, CameraManager, ChessboardDetector	摄像头采集、棋盘定位、整盘识别、多帧稳定与动态走子推断
core	RTMPOSE_ONNX, FULL_CLASSIFIER_ONNX, ChessboardDetector	RTMPose 四角关键点与全局分类器的 ONNX 推理核心
ai	AIEngine	Pikafish UCI 引擎封装 (uci/position/go/bestmove 解析)
robot	RobotPersistentClient, BoardToArmConfig, uci_to_arm_command	五值指令协议、持久化 TCP 客户端、UCI 到机械臂指令映射
game_manager	GameManager	无界面 (仿真) 对弈主循环编排
web_simulation	app, domain, robot_cmd, game_logic	Flask 组合根、纯棋局函数、机器人指令封装、状态机与路由
config.py	模块级常量	摄像头、棋盘、引擎、机器人网络与标定的集中配置

2.3 运行时组合与依赖关系

系统的可测试性不仅来自“纯函数包 + 职责包”的静态划分，更来自运行时的组合根模式。web_simulation/app.py 作为组合根，持有可变的共享 game_state 字典与若干长生命周期组件（识别器、引擎、机器人控制器、TCP 客户端）；这些组件并非在导入时实例化，而是由 services.py 中的工厂以懒加载单例方式创建——例如 get_recognizer 保证并发请求只构造一个识别器实例，get_robot_tcp_client 在主机/端口变化时才重建持久连接。这种“依赖在使用处解析、实例全局唯一”的安排，使单元测试可以分别替换任意一层：视觉测试可注入合成的棋盘帧，协议测试可直接构造 RobotArmCommand 而无需真实 TCP，领域逻辑测试可在无浏览器环境下覆盖纯函数。图 2.2 给出组合根与各层的依赖关系。

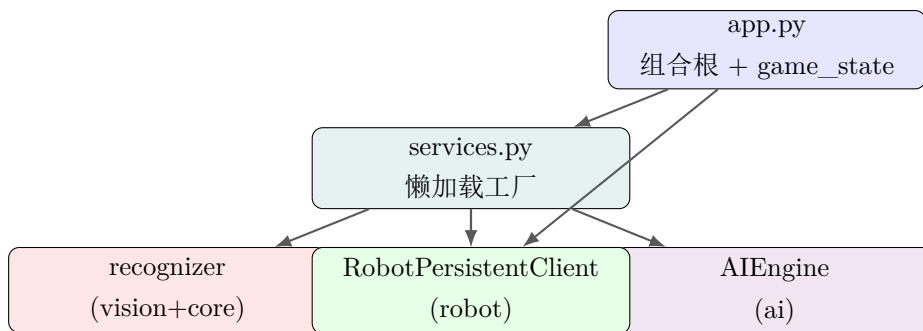


图 2.2: 组合根 (app.py) 通过 services.py 的懒加载工厂解析各层依赖; domain.py 为不依赖 I/O 的纯函数, 便于独立测试。

本章小结

本章确立了“感知—认知—执行—接口”四层架构, 定义了层间以 FEN、五值指令与 ACK 报文为载体的数据契约, 并以模块职责表与组合根依赖图说明系统如何通过纯函数包与懒加载单例实现解耦与可测试性。后续章节将沿数据流方向, 自感知层逐层向下展开。

第三章 视觉感知与棋盘识别

视觉子系统负责把“一张桌子上的物理棋盘”转换为机器可读的 FEN 字符串。其流水线如图 3.1 所示，包含图像采集、棋盘定位、整盘分类、FEN 生成与多帧稳定五个阶段。

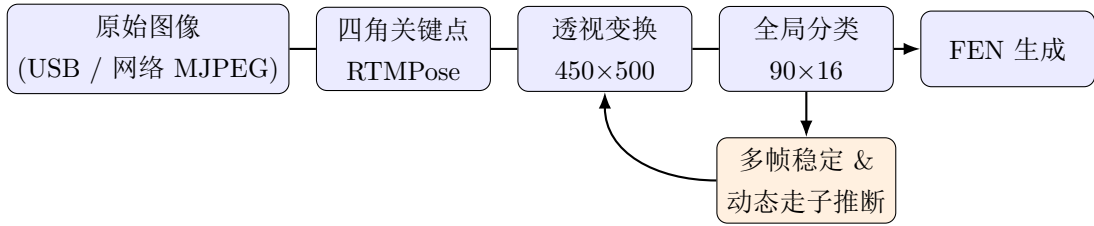


图 3.1: 视觉识别流水线：从原始图像经四角关键点定位、透视校正，到全局分类与 FEN 生成，再由多帧稳定模块对连续帧去噪并推断人类走子。

3.1 图像采集与摄像机模型

系统支持两类图像来源：本地 USB 摄像头（通过 OpenCV VideoCapture 采集）与网络摄像头（如 Orange Pi / Raspberry Pi 提供的 MJPEG 快照或流，由 HttpSnapshotCapture 用 urllib 抓取 JPEG/MJPEG，并以 SOI/EOI (0xFFD8/0xFFD9) 切帧^[13]。采集参数由 config.py 集中配置：分辨率 640×480 、帧率 30 FPS、设备索引 1，并可在启动时通过 startup.py 覆盖。识别前若图像宽度超过 CHRO_RECOGNITION_MAX_WIDTH（默认 960 px），会先按比例下采样以降低推理负担。

从几何上看，摄像头成像可建模为针孔模型：三维世界点 $\mathbf{X} = [X, Y, Z]^T$ 经投影矩阵 $\mathbf{P}$ 映射到图像像素 $\mathbf{x} \sim \mathbf{P}\mathbf{X}$ 。当摄像头近似垂直俯视棋盘时，棋盘平面与像平面近似为一般位置的两平面，其间的映射可统一用一个 3×3 单应矩阵（homography） $\mathbf{H}$ 描述（详见下一节）。这一建模是透视校正的理论基础^[25]。

3.2 棋盘定位：四角关键点与透视变换

与依赖规则棋盘纹理的 findChessboardCorners 不同，本系统使用一个定制 RTMPose ONNX 模型，直接预测棋盘的四个角点（记为 A0、A8、J0、J8，分别对应左上、右上、左下、右下交叉点）^[14,16]。RTMPose 的底层采用 SimCC（Simple Coordinate Classification）关键点范式：它不回归高斯热图峰值，而是将两个坐标轴上的位置各自离散为 512 个 bin 进行分类，输出形状为 4×512 的“坐标分类”张量，再以 argmax 还原亚像素坐标。该范式在约束场景下对小目标角点尤为稳定，且推理开销低。

四点经有效性门限（形状为 (4, 2)、四角置信度均不低于 0.20、边距容差 80 像素）校验后，通过单应矩阵将原始图像校正为归一化俯视图。

3.2.1 单应矩阵与透视校正原理

设棋盘平面上的源点 $\mathbf{x}_s = (u_s, v_s, 1)^\top$ 与俯视图上的目标点 $\mathbf{x}_d = (u_d, v_d, 1)^\top$ 满足

$$\lambda \mathbf{x}_d = \mathbf{H} \mathbf{x}_s, \quad \mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}, \quad (3.1)$$

其中 λ 为尺度因子， $\mathbf{H}$ 含 8 个自由度（在归一化 $h_{33} = 1$ 下）。已知四对对应点即可列出 8 个线性方程唯一求解 $\mathbf{H}$ ——这就是经典的直接线性变换（DLT）闭式解。系统调用 OpenCV 的 `getPerspectiveTransform`（其内部即 DLT 求解）完成该过程，并将原始图像以 `warpPerspective` 校正为 450×500 像素的俯视图（目标角点带 50 px 内边距，故每格约 50 像素）^[25]。由于姿态模型推理开销较大，系统对关键点结果进行缓存（默认刷新间隔 8.0 s 或最多 120 帧），在不移动摄像头的前提下避免每帧重算。

3.3 整盘棋子分类

校正后的整张棋盘图像被一次性送入一个全局分类器 ONNX 模型（FULL_CLASSIFIER_ONNX），输出形状为 90×16 的张量，即 10 行 $\times$ 9 列共 90 个交叉点，每个点对应一个 16 维 softmax。这 16 个类别包含“空”（.）、“未识别”（x）以及红黑双方的 14 种棋子，具体映射如表 3.1 所示。相比逐格裁剪分类或通用目标检测器（如 YOLO^[17]），整盘一次性推理显著降低了调度与后处理开销，也更契合嵌入式/边缘部署场景。默认模型 `nano_v3-0319.onnx` 的预处理为先中心裁剪至 400×450 再缩放至 280×315 ，采用 ImageNet 均值/方差归一化^[18-19]。

表 3.1: 全局分类器输出的 16 个类别（红方大写、黑方小写）

符号	含义	符号	含义	符号	含义	符号	含义
.	空	x	未识别	K/k	帅/将	A/a	仕/士
B/b	相/象	N/n	马	R/r	车	C/c	炮
P/p	兵/卒						

3.4 多帧稳定与动态走子推断

单帧识别存在抖动与误检。系统以 `StableBoardBuffer` 维护最近 N 帧（默认窗口 `STABLE_WINDOW=5`）的棋盘投票，某交叉点需出现在不少于 $\lceil N \times \text{STABLE_RATIO} \rceil$ 帧（`STABLE_RATIO=0.60`）方判定为“稳定”，其中“空”与“未识别”不参与计数。在此之上，`DynamicBoardTracker` 以首张稳定棋盘为基线，通过比较占用变化推断人类的一步走子，其内部状态机如图 3.2 所示：初始处于 `observing`，首次锁定后进入 `initial_locked`；之后每次与基

线比较，归一为“未变化 (unchanged)”“一步走子 (move)”或“非法变化 (invalid_change)”。正常走子对应“一处消失 + 一处出现”；吃子对应“一处消失 + 一处改变”。为过滤噪声，系统内置基于中国象棋规则的合法性校验 (`_is_legal_xiangqi_move`，覆盖兵/炮/马/象/士/将的走法与“蹩马腿”“塞象眼”等约束)，并对高噪声情形做容错处理（允许少量可忽略的噪声点，按距离与噪声数打分择优）。

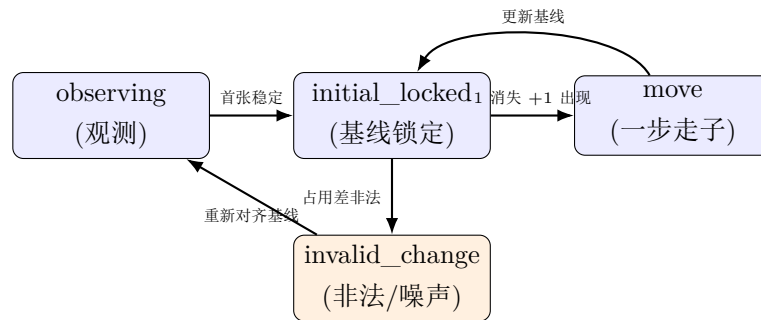


图 3.2: DynamicBoardTracker 的状态机：以首张稳定棋盘为基线，依据占用变化推断走子或判为噪声。

3.5 真实棋盘图像示例

图 3.3 给出系统在实际部署中采集的一张棋盘图像（已用于调试棋盘定位与分类），可见摄像头俯视视角、棋盘纹理与棋子排布。该图像经前述流水线处理后即可生成标准开局 FEN。



图 3.3: 系统采集的真实棋盘图像示例（调试用）。经 RTMPose 四角定位与透视校正后送入全局分类器。

本章小结

本章完整给出了视觉子系统的技术路线：从针孔成像建模出发，用 RTMPose/SimCC 四角关键点与 DLT 单应完成透视校正，以单一全局分类器一次性输出整盘 90 个交叉点类别，并通过多帧投票与基于规则的动态走子推断抑制单帧噪声。该方案以“定位 + 整盘分类”替代“逐格检测 + 目标检测器”，在边缘算力下兼顾了准确率与延迟。

第四章 棋局决策引擎与棋局表示

本章说明认知层如何把感知层给出的 FEN 转化为一步推荐着法。需要强调的是：本系统不实现搜索与评估算法，而是作为 UCI 客户端驱动 Pikafish（Stockfish 的中国象棋分支）；alpha-beta 剪枝、迭代加深、静态搜索与 NNUE 神经评估均位于所调用引擎内部。下文先给出本系统的协议驱动与结果解析职责，再对引擎所采用的搜索原理作背景性说明。

4.1 Pikafish 与 UCI 协议

认知层通过 UCI 协议驱动 Pikafish 引擎^[2,9]。AIEngine 的启动序列为：发送 `uci` 并等待 `uciok`；设置 `UCI_Variant value xiangqi` 以启用中国象棋变体；设置 `Skill Level`（取自 `ENGINE_DEPTH=15`）与可选的 `Hash (HASH_SIZE_MB=128)`。此处 `ENGINE_DEPTH` 同时被复用为“棋力等级”与“搜索深度”两处参数，是本系统一处务实但需留意的设计约定。决策时，引擎以 `position fen <FEN> [moves ...]` 载入局面，再以 `go depth 15` 搜索，解析 `stdout` 中的 `bestmove` 作为推荐着法；`analyze_position` 则改用 `go movetime` 并解析 `info ... score cp/depth/pv` 以呈现评分与主要变化。默认引擎二进制为 `Pikafish/pikafish-avx2.exe`，配套 NNUE 权重约 50.8 MiB。Web 端 `/api/ai_move` 出于响应速度考虑，默认以较轻的 `go depth 8` 调用。

4.2 搜索引擎原理（引擎侧）

作为背景，现代棋类引擎的搜索以极小化极大（**minimax**）框架为基础：在博弈树上交替最大化己方、最小化对方收益。为在有限时间内深入搜索，引擎普遍采用以下技术^[8]：

- **alpha-beta 剪枝**：在遍历博弈树时维护 α （己方已可保证的下界）与 β （对方已可保证的上界），一旦区间为空即剪掉该子树，可将有效分支因子由 b^d 降至约 $b^{d/2}$ 。
- **迭代加深（iterative deepening）**：从浅层逐步加深搜索深度，既保证随时可返回最佳着法，又能以浅层结果排序引导深层移动顺序、提升剪枝效率。
- **静态搜索（quiescence search）**：在叶子节点继续搜索吃子等“不稳定”着法，抑制水平效应（horizon effect）带来的评估抖动。
- **NNUE 神经评估**：以增量更新的半接（half-KP）特征配合小型神经网络评估局面，取代手工评估函数，显著提升棋力^[12]。本系统 NNUE 权重约 50.8 MiB，由引擎在启动时载入。

上述算法由 Pikafish 在 UCI 协议内部完成；本系统只负责设参与解析 `info / bestmove`，从而把“强引擎”作为可替换的能力模块接入系统。

4.3 中国象棋 FEN 与坐标表示

系统采用扩展 FEN 表示棋局，开局串为

```
rnbakabnr/9/1c5c1/p1p1p1p1p/9/9/P1P1P1P1P/1C5C1/9/RNBAKABNR w - - 0 1
```

棋盘为 10 行 × 9 列，每方 7 类棋子（车/马/相(象)/士(仕)/帅(将)/炮/兵(卒)）。FEN 的首行对应棋盘第 9 行（黑方底线），末行对应第 0 行（红方底线）；走子方用 w/b 表示，本系统中 w 代表红方先行^[1]。FENUtils 提供 `parse_fen`、`to_fen` 与 `extract_move_from_fen_change`（通过对比前后两局 FEN 还原一步 UCI 着法），是连接视觉识别与决策的核心数据契约。棋子字符与基础属性如表 4.1 所示。

表 4.1: 中国象棋 FEN 棋子字符与基础属性							
	车	马	相/象	士/仕	帅/将	炮	兵/卒
红方	R	N	B	A	K	C	P
黑方	r	n	b	a	k	c	p
棋盘	9 列 × 10 行，含河界与九宫						

4.4 着法记谱转换

为便于人机界面展示，MoveNotationUtils 在 UCI 坐标记法（如 h2e2）与中文/简易 WXF 记法间互转。UCI 以“起点列行 + 终点列行”编码；中文记法则以“棋子名 + 起点路数/线数 + 进退/平 + 终点”表达（如“炮二平五”），二者通过 `xq_utils/coordinates.py` 中的列行索引换算衔接。

本章小结

本章澄清了认知层的职责边界：本系统是 Pikafish 的 UCI 客户端，承担启动、设参与 `bestmove/ info` 解析，而非自行实现搜索；文中对 alpha-beta、迭代加深、静态搜索与 NNUE 的说明均指向引擎内部实现。扩展 FEN 与记谱转换构成了感知层与认知层之间的标准数据契约。

第五章 机械臂控制与通信协议

5.1 五值 ASCII 指令协议

执行层定义了一种极简的五值指令协议：上位机每步仅下发五个以逗号分隔的整数

`startX, startY, endX, endY, signal`

其中 `signal=0` 表示普通移动, `signal=1` 表示吃子(抓取目标位已有棋子)。所有角度解算与气泵时序均在 STM32 下位机侧完成。该协议经 **TCP 端口 8086** 下发, 默认主机为 `192.168.0.102`; 每条指令以 `CRLF (\r\n)` 成帧。协议字段语义如表 5.1 所示。极小的协议表面积使上位机逻辑简单、易于测试(协议测试直接断言几何与 ACK)。

表 5.1: 五值指令协议字段语义

字段	含义
<code>startX, startY</code>	起点在机械臂坐标系下的 x, y (毫米)
<code>endX, endY</code>	终点在机械臂坐标系下的 x, y (毫米)
<code>signal</code>	0= 普通移动; 1= 吃子(需先移走被吃子); 99= 归位(<code>startX/startY</code> 解释为 M1/M2 相对角)
帧格式	以 <code>CRLF</code> 结尾; 归位指令为 <code>-17.1848,-55.6304,0,0,99</code>
完成确认	下位机回复含 <code>STATE:5</code> 且 <code>RESULT:1</code> (归位另含 <code>CMD:99</code>) 表示动作完成

5.2 棋盘坐标到机械臂坐标的映射

机械臂坐标系以棋盘右上角交叉点(黑方右车位)为原点。设第 f 列 ($0 \leq f \leq 8$)、第 r 行 ($0 \leq r \leq 9$, $r = 9$ 为黑方底线)的交叉点到原点的映射由 `BoardToArmConfig.point_for_square` 实现, 其核心逻辑为

Listing 5.1: 坐标映射核心(节选自 `robot/protocol.py`)

```
vertical_intervals = 9 - rank
river_extra = (river_spacing_mm - rank_spacing_mm) if rank <= 4 else 0
return (
    origin_x + (8 - file_index) * file_spacing_mm,
    origin_y + vertical_intervals * rank_spacing_mm + river_extra,
```

代入列间距 $s_f = 34 \text{ mm}$ 、普通行间距 $s_r = 30 \text{ mm}$ 、河界行间距 $s_{\text{river}} = 32 \text{ mm}$ ，可得棋盘总宽 $34 \times 8 = 272 \text{ mm}$ ；纵向因红方半场（第 0–4 行）每行叠加 $+2 \text{ mm}$ 的河界间隔，使跨越河界的唯一一段间距为 32 mm ，总高亦为 $30 \times 8 + 2 = 272 \text{ mm}$ ，即 $272 \times 272 \text{ mm}$ 的正方形盘面（图 5.1）。该几何与固件文档的测量值一致，是上位机与下位机对齐的物理基础。

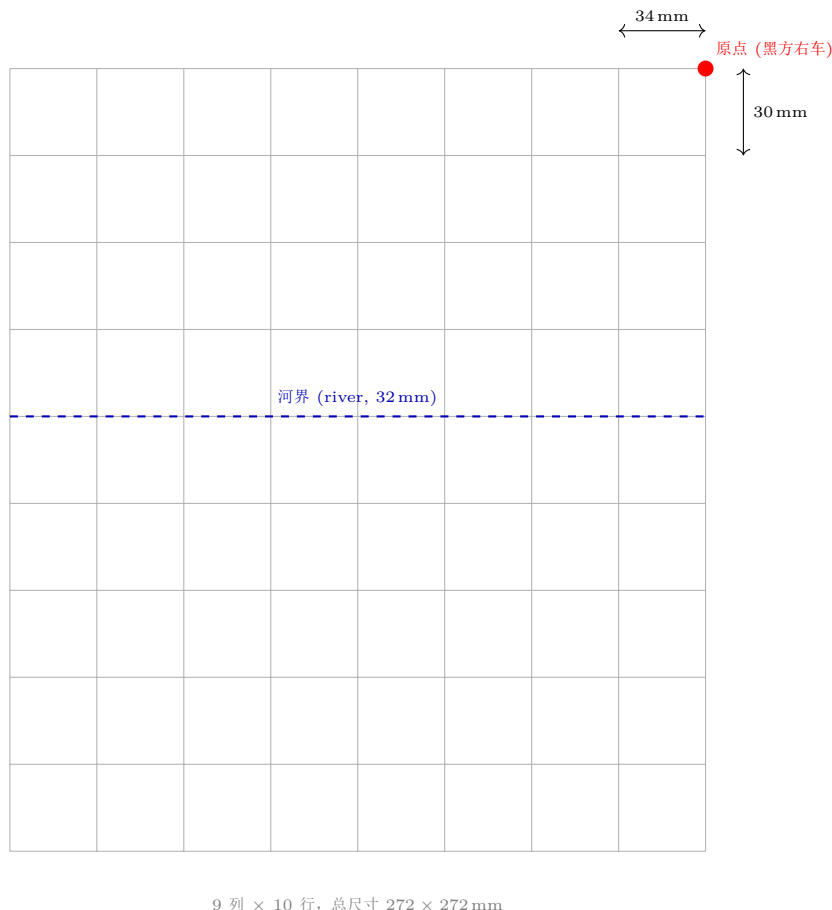


图 5.1: 棋盘几何与坐标映射示意。原点位于右上角（黑方右车），列间距 34 mm 、普通行间距 30 mm 、河界行间距 32 mm 。

5.3 持久化 TCP 客户端与确认握手

`RobotPersistentClient` 以 `socket.create_connection` 建立长连接，并设置 `SO_KEEPALIVE` 与 `TCP_NODELAY`。每条指令以 `CRLF` 成帧；`send_robot_command` 阻塞直至收到 `motion_acknowledged`（要求响应同时含 `STATE:5` 与 `RESULT:1`，且不含 `CMD:99`），由于运动指令非幂等，客户端在缺失 `ACK` 时不自动重发而直接报错。归位指令 `send_homing_command` 发送 `-17.1848, -55.6304, 0, 0, 99`（`signal=99` 标记归位，`startX/startY` 被下位机解释为 `M1/M2` 相对转角，单位为度），并等待 `homing_acknowledged`（要求 `STATE:5, RESULT:1` 且该回执含 `CMD:99`）。Web 端在硬件模式下启动对局前必须先完成归位握手，否则以 `HTTP 503` 拒绝开局。

上位机与 STM32 之间的典型交互时序如图 5.2 所示。ACK 解析同时支持“仅 1”与“STATE:5,RESULT:1”两种旧/新固件回执格式，并严格以 CMD:99 区分归位与普通走子。

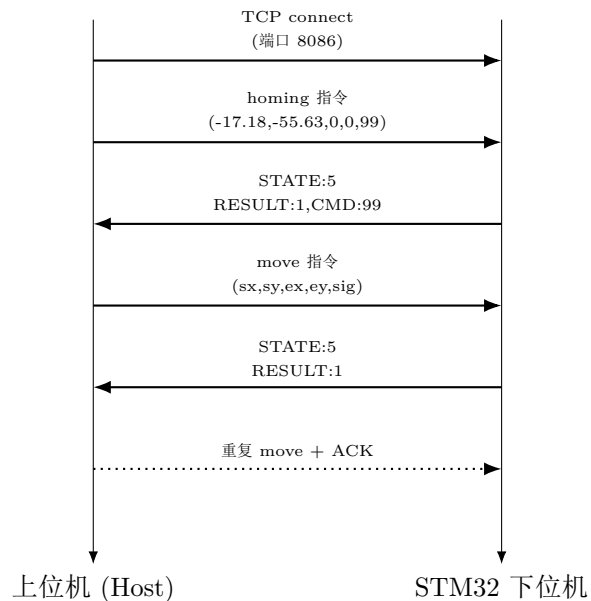


图 5.2: 上位机与 STM32 的通信时序：连接后先归位握手，再逐条下发五值移子指令并等待完成确认。

5.4 STM32 下位机实现与运动学

激活的固件位于 `firmware/stm32-f103-keil-robot`: ESP8266 在 TCP 8086 上监听，解析 +IPD, 后的数据，并以 `sscanf(data,"%d,%d,%d,%d,%d",&x1,&y1,&x2,&y2,&sig)` 提取五整数后写入 RobotCmd 结构体^[24]。归位时 `signal=99`，下位机将 `startX/startY` 直接作为 M1/M2 相对转角（度）执行；普通走子则进入 SCARA 运动学解算。

以平面 SCARA 型机械臂为例，其正向运动学为

$$\begin{aligned} x &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), \\ y &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2), \end{aligned} \quad (5.1)$$

下位机依据目标 (x, y) 反解关节角 (θ_1, θ_2) 。`ik.c` 中的实际逆解（先平移到基座局部坐标 $x_l = x - \text{base}_x$, $y_l = y - \text{base}_y$ ）为

Listing 5.2: SCARA 逆运动学（节选自 `firmware/stm32-f103-keil-robot/ik.c`）

```

float r_sq = x_l*x_l + y_l*y_l;
float cos_q2 = (r_sq - L1*L1 - L2*L2) / (2*L1*L2);
cos_q2 = clamp(cos_q2, -1, 1);    // 越界置错误位
q2 = acosf(cos_q2);
term1 = atan2f(y_l, x_l);
term2 = atan2f(L2*sinf(q2), L1 + L2*cosf(q2));
q1 = term1 - term2;              // 肩角 - 肘角
  
```

即标准两连杆平面 SCARA 逆解：由余弦定理求肘角、由几何关系求肩角。本系统实例化参数为 $L_1 = L_2 = 220$ mm、基座偏移 $\text{base}_x = 136$ mm、 $\text{base}_y = -100$ mm。解算得到关节角后，由定时器输出 PWM 驱动步进/伺服电机，并以 GPIO 控制气泵电磁阀完成“吸起—平移—放下”的移子动作^[22]。气泵由 TIM2 产生 50 Hz（周期 20 ms）PWM，脉宽 500 μs 为关、2500 μs 为开（标准电磁阀/舵机脉宽），经 Pump_On/Off 与 Valve_On/Off 切换完成抓取时序。

系统中另保留一个已弃用的 JSON 传输栈（robot/legacy_tcp_client.py 经 TCP 端口 5000 下发 JSON，配套 firmware/stm32-tcp-server），它不再被主程序使用，仅作为历史固件的兼容客户端保留。论文明确区分二者，以免混淆激活路径。

本章小结

本章给出了执行层的完整技术细节：以五值 ASCII 协议与 CRLF 成帧实现极小表面积的指令通道；以右上角原点 + 34/30/32 mm 几何完成棋盘到机械臂的精确映射；以持久化 TCP 客户端与 STATE:5,RESULT:1（归位另含 CMD:99）握手保证确定性，并坚持“非幂等指令缺失 ACK 即上报、不重发”的可靠原则；下位机在 STM32+ESP8266 上以 SCARA 逆运动学与 50 Hz PWM 气泵时序完成物理移子。文中还区分了激活的五值 8086 栈与已弃用的 JSON 5000 栈。

第六章 软件编排与 Web 仿真

6.1 Flask 组合根与蓝图架构

`web_simulation` 是系统的激活控制路径, 以 Flask 在端口 5000 提供服务^[26]。`app.py` 作为组合根, 持有可变的共享 `game_state` 字典与若干长生命周期组件 (识别器、引擎、机器人控制器、TCP 客户端), 并通过蓝图将路由拆分为 `status`、`camera`、`recognition` 与 `game` 四个职责域。纯棋局函数集中在 `domain.py` (如 `board_state_to_fen`、`apply_uci_to_board_state`), 机器人指令封装在 `robot_cmd.py`, 状态机与硬件/仿真切换在 `game_logic.py`。该结构使 Web 层与核心算法解耦, 便于在无浏览器环境下以单元测试覆盖领域逻辑。

6.2 领域纯函数与状态表示

`domain.py` 将棋盘表示为键为 "col,row" 的字典 (红方棋子大写、黑方小写, `row=0` 在黑方底线), 并提供:

- `board_state_to_fen`: 把 10×9 字典压缩为标准 FEN (含走子方与半回合);
- `apply_uci_to_board_state`: 把一步 UCI 应用到字典并返回是否发生吃子;
- `board_pos_to_uci`: 把 "col,row" 数组坐标转回 UCI (`f"UCI_FILES[col]9-row"`);
- `color_to_turn_char` / `opposite_color` 等走子方工具。

这些纯函数无 I/O 副作用, 是视觉、决策、执行三层共享的“词汇表”, 也是测试覆盖的重点。

6.3 对局状态机与双模式

系统支持硬件与仿真两种机器人模式 (Web 默认硬件)。核心流程 `apply_ai_best_move` 执行以下步骤:

- (1) 将 AI 走法转为五值命令并格式化, 构造去重令牌 `ai_command_token`;
- (2) 重复走法保护: 若尾步或令牌重复, 直接忽略而不重发;
- (3) 预应用: 把走法写入 `move_history`、更新 `board_state` 与 FEN, 并暂停视觉识别 (`vision_pause_until`, 默认 15s);
- (4) 分发: 仿真模式直接返回成功回执; 硬件模式经持久 TCP 客户端下发并等待 `STATE:5,RESULT:1`;

(5) 硬件 ACK 后立即复位动态基线并恢复视觉识别，仿真模式则等暂停期满恢复。

此外，`/api/game/start` 在硬件模式下会先发归位指令并等待 `homing ack`，失败则返回 503 且不开局；仿真模式跳过归位。该“开局前归位”契约由前端与后端测试共同固化。

6.4 HTTP API 与前端交互

四个蓝图暴露的主要能力包括：`status` (`/api/status` 完整状态、`/api/robot/status` 探测 STM32)、`camera` (`/api/camera/stream` 以 `multipart/x-mixed-replace` 推送 MJPEG 实时流、`/api/capture` 单帧)、`recognition` (`/api/recognize` 静态识别、`/api/recognize/dynamic` 动态识别主循环、`/api/ai_move` 后台思考并应用走法)、`game` (`/api/game/start`、`/api/game/reset`)。前端 `static/js/app.js` 通过轮询 `/api/status` 与 `/api/ai_status`、订阅 MJPEG 流，实现实时视频、评分 (`depth/score/pv/nodes`)、走子历史与机器人连接状态的统一可视化。

本章小结

本章说明接口层如何用 Flask 组合根 + 四个蓝图把核心算法“缝合”为人机界面：领域纯函数充当无副作用的共享词汇表，对局状态机实现硬件/仿真双模式与“开局前归位”“重复走法不重发”“ACK 后即时恢复视觉”等关键契约，MJPEG 流与轮询式状态接口则提供了低延迟的交互通道。

第七章 工程实现与质量保障

7.1 代码规模与解耦

重构后的系统以“纯函数包 + 职责包 + 组合根”组织，便于单元测试与替换。各包的代码规模如表 7.1 所示（不含注释与空行，固件为 C/C++ 源）。整库约 6 500 行 Python 与 6 400 行固件代码。

表 7.1: 主要模块代码规模（实测）

模块	Python 文件数	代码行数（不含注释/空行）
xq_utils	5	448
cli	3	261
game_manager	4	445
vision	9	1 397
ai	2	324
robot	4	921
core	7	821
web_simulation	12	1 933
config.py	1	115
小计（Python）	47	约 6 665
firmware（C/C++）	27	6 448

7.2 测试体系

系统在 `tests/` 下提供 10 个测试模块、共 76 个测试方法，覆盖摄像头与图像来源、稳定器与识别并发、五值协议与 Web 后端循环、前端归位契约、STM32 归位契约与 Web 领域函数等。最近一次运行结果为 75 项通过、1 项未通过——该失败源于运行环境的一个特定限制（某 Windows 环境变量超过 32 767 字符导致 `mock.patch.dict` 异常），与识别/协议逻辑无关。协议与后端循环测试（约占全部测试方法的一半）固化了如表 7.2 所示的关键不变量。

表 7.2: 测试固化的关键不变量（节选）

不变量	说明
协议几何	五值命令由 BoardToArmConfig (34/30/32mm, 右上角原点) 精确推导; 过河用 32mm 纵向间距
吃子信号	由移动前目标格占用决定 (_target_has_piece)
归位共享客户端	归位与走子复用同一持久 socket; /api/game/start 先归位再开局
无自动重发	运动指令缺失 ACK 即上报, 不重试 (指令非幂等)
重复走法防护	尾步重复与重复命令令牌不重发
双模式契约	硬件开始必须归位成功才开局; 仿真跳过归位
动态基线	机器臂落子后需连续 N 帧匹配才锁定, ACK 后即时恢复识别

7.3 ONNX 推理与执行提供者

core/runonnx 的推理后端按优先级选择 CUDAXecutionProvider → DmlExecutionProvider → CPUExecutionProvider^[21]。在 Windows 上, 后端先探测 cublas64_12.dll、cublasLt64_12.dll、cudnn64_9.dll (即 CUDA 12 + cuDNN 9) 是否存在, 缺失则回退 CPU; 若 GPU 会话初始化失败, 还会捕获异常二次回退。会话启用 ORT_ENABLE_ALL 图优化, 并保证在无 GPU 环境下仍可运行 (仅推理速度下降)。

本章小结

本章以代码规模表、测试不变量表与 ONNX 执行提供者回退链说明工程实现的质量保障: 系统以约 6665 行 Python 与 6448 行固件实现, 并辅以 76 个测试方法固化协议几何、吃子信号、归位契约、无自动重发等关键不变量, 确保视觉、决策、控制三层可独立回归。

第八章 实验与系统指标

本节汇总可在代码仓库中**直接核验**的系统指标。我们刻意避免给出未经真实硬件测量的精度或时延数字，以保证结论的可复现性；涉及性能的下界/上界以“设计特征”或“规格参数”明确标注。

模型规格方面，两个默认 ONNX 模型与 NNUE 权重的体积与张量形状如表 8.1 所示，均为仓库内真实文件。配置常量方面，表 8.2 列出与识别、决策、控制相关的关键参数（取自 `config.py`）。

表 8.1: 模型与权重规格（仓库真实文件）

文件	角色	大小	输入	输出
4_v6-0301.onnx	RTMPose 四角关键点	≈ 10.2 MiB	256×256	SimCC $4 \times 512(x,y)$
nano_v3-0319.onnx	全局棋盘分类器	≈ 29.7 MiB	280×315	90×16
pikafish.nnue	NNUE 评估权重	≈ 50.8 MiB	—	—

表 8.2: 关键配置常量（取自 `config.py`）

常量	取值
摄像头	CAMERA_WIDTH=640, CAMERA_HEIGHT=480, CAMERA_FPS=30
棋盘	BOARD_ROWS=10, BOARD_COLS=9
稳定	STABLE_WINDOW=5, STABLE_RATIO=0.60, AUTO_STABLE_FRAMES=25
引擎	ENGINE_DEPTH=15, THINK_TIME=5000 ms, HASH_SIZE_MB=128
机器人网络	ROBOT_NETWORK_PORT=8086, ROBOT_NETWORK_HOST=192.168.0.102
机械臂几何	$s_f=34$ mm, $s_r=30$ mm, $s_{\text{river}}=32$ mm, 原点右上角
归位	ROBOT_HOMING_M1= -17.1848° , ROBOT_HOMING_M2= -55.6304°
抓取高度	GRIPPER_PICK_HEIGHT=80, GRASP_HEIGHT=10, MOVE_HEIGHT=50
移动速度	ROBOT_SPEED_FAST=100, ROBOT_SPEED_SLOW=30

本章小结

本章仅给出可在仓库直接核验的模型规格、配置常量与代码/测试规模，刻意不引入未经真实硬件测量的精度或时延数据，以保证论文结论可复现。这些规格共同刻画了系统的设计特征

与资源占用。

第九章 结论与展望

本文介绍了 CH-RO 中国象棋对弈机器人系统，覆盖从视觉棋盘识别、Pikafish UCI 决策到 STM32 五值指令执行的完整闭环，并以 Flask Web 界面贯通人机交互。系统通过“RTMPose 四角关键点 + 全局分类器”的轻量识别方案、确定性的五值指令与完成确认握手，以及硬件/仿真双模式，在解耦、可测试的代码架构下实现了端到端对弈。

需要如实说明两点工程现实：其一是代码库中仍并存两套传输栈（激活的五值 8086 栈与已弃用的 JSON 5000 栈），主程序仅使用前者；其二是仿真控制器与硬件路径在坐标/几何约定上存在历史演进差异，属于应在重构时统一的遗留问题。论文对此予以显式标注，以避免误导后续维护者。

未来工作包括：(1) 统一坐标与几何约定，消除仿真/硬件路径差异；(2) 在真实硬件上建立识别准确率与移子时延的基准评测；(3) 引入更轻量的分类/关键点模型以提升边缘实时性；(4) 扩展开局库、残局表与更强的时限/难度控制，丰富对弈体验。

参考文献

- [1] Wikipedia. Xiangqi[EB/OL]. 2024 [2026-08-03]. <https://en.wikipedia.org/wiki/Xiangqi>.
- [2] 开发团队 P. Pikafish: 免费且强大的 UCI 中国象棋引擎[EB/OL]. 2024 [2026-08-03]. <https://github.com/Pikafish/pikafish>.
- [3] 开发团队 S. Stockfish: 开源国际象棋引擎[EB/OL]. 2024 [2026-08-03]. <https://stockfishchess.org>.
- [4] CAMPBELL M, HOANE A J, HSU F H. Deep Blue[J]. 2002, 134(1-2)-. Campbell, Murray, 2002. DOI: [10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1).
- [5] TESAURO G. Temporal difference learning and TD-Gammon[J]. 1995, 38(3)-. Tesauro, Gerald, 1995. DOI: [10.1145/203330.203343](https://doi.org/10.1145/203330.203343).
- [6] BAXTER J, TRIDGELL A, WEAVER L. KnightCap: a chess program that learns by combining TD(λ) with game-tree search[C]//Proceedings of the 15th International Conference on Machine Learning (ICML 1998). 1998: 61-68.
- [7] SILVER D, HUBERT T, SCHRITTWIESER J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. 2018, 362(6419)-. Silver, David, 2018. DOI: [10.1126/science.aar6404](https://doi.org/10.1126/science.aar6404).
- [8] RUSSELL S, NORVIG P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Hoboken: Pearson, 2020.
- [9] MEYER-KAHLEN S, CHIDESTER S. UCI (Universal Chess Interface) protocol[EB/OL]. 2004 [2026-08-03]. <https://www.shredderchess.com/chess-info/features/uci-universal-chess-interface.html>.
- [10] 刘建宇. UCCI (Universal Chinese Chess Interface) 协议[EB/OL]. 2007 [2026-08-03]. <https://www.xqbase.com/protocol/ucci.htm>.
- [11] 刘建宇. XQFEN: 中国象棋扩展棋局表示法[EB/OL]. 2007 [2026-08-03]. <https://www.xqbase.com/protocol/cchessfen.htm>.
- [12] NASU Y. Efficiently updatable neural-network-based evaluation function for shogi[C]//Proceedings of the 28th Conference of the International Computer Games Association (ICGA 2018). 2018.
- [13] BRADSKI G. The OpenCV Library[J]. 2000, 25(11)-. Bradski, G., 2000.

- [14] JIANG T, LU P, ZHANG L, et al. RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose[J]. 2023()--. Jiang, Tao, 2023. arXiv: [2303.07399](https://arxiv.org/abs/2303.07399).
- [15] OpenMMLab. MMPose: 基于 PyTorch 的开源姿态估计工具箱[EB/OL]. 2023 [2026-08-03]. <https://github.com/open-mmlab/mmpose>.
- [16] LI Y, YANG S, LIU P, et al. SimCC: a Simple Coordinate Classification Perspective for Human Pose Estimation[C]//Proceedings of the European Conference on Computer Vision (ECCV 2022). 2022: 89-106. arXiv: [2107.02355](https://arxiv.org/abs/2107.02355).
- [17] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). 2016: 779-788. arXiv: [1506.02640](https://arxiv.org/abs/1506.02640).
- [18] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. ImageNet Classification with Deep Convolutional Neural Networks[C]//Advances in Neural Information Processing Systems (NeurIPS 2012): vol. 25. 2012: 1097-1105.
- [19] DENG J, DONG W, SOCHER R, et al. ImageNet: A Large-Scale Hierarchical Image Database[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2009). 2009: 248-255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [20] BAI J, LU C, ZHANG K, et al. ONNX: Open Neural Network Exchange[J]. 2019()--. Bai, Junjie, 2019. arXiv: [1907.07064](https://arxiv.org/abs/1907.07064).
- [21] Microsoft. ONNX Runtime: 跨平台高性能推理引擎[EB/OL]. 2024 [2026-08-03]. <https://onnxruntime.ai>.
- [22] CRAIG J J. Introduction to Robotics: Mechanics and Control[M]. 4th ed. New York: Pearson, 2017.
- [23] SICILIANO B, KHATIB O. Springer Handbook of Robotics[M]. 2nd ed. Berlin: Springer, 2016.
- [24] STMicroelectronics. STM32F103xx 增强型系列参考手册 (RM0008)[EB/OL]. 2023 [2026-08-03]. https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [25] HARTLEY R, ZISSERMAN A. Multiple View Geometry in Computer Vision[M]. 2nd ed. Cambridge: Cambridge University Press, 2004.
- [26] Pallets. Flask: 轻量级 WSGI Web 应用框架[EB/OL]. 2024 [2026-08-03]. <https://flask.palletsprojects.com>.